FCV Lab 11: Mini Project (Phase II)

Vision-Assisted Survivor Localisation in Maritime Search and Rescue using Texture-Anomaly Modelling

Final Report

Team Members

Name	Registration no.
Tanish Hemant Kuvar	240962540
M. Vishal Kumar	240962546
Samyak Jain	240962638

Section AIML-A

November 2026

Abstract

This project finds people in the water from aerial RGB imagery and reached 92.4% recall@5 on a synthetic benchmark, using only classical computer vision and no training data. People in the water often cover fewer than 16 pixels, too few for appearance-based recognition. The system therefore models the water rather than the person.

Each frame passes through six stages: illumination normalisation, Gabor and uniform LBP texture description, chi-square anomaly scoring, glint suppression, morphological shape filtering and temporal tracking. The output is a ranked shortlist of at most k regions per frame, drawn as an overlay in a Tkinter operator GUI.

On 6 synthetic sequences (60 frames each, 739 annotated people), the full system scored 51.7% precision and 1.77 false alarms per frame at $k = 5$. Recall rose to 99.3% once the tracker had warmed up. Throughput was about 2.7 frames/s at 1280×720 on a laptop CPU. These numbers validate the pipeline end to end; they do not measure real-world performance, which still has to be tested on public datasets.

Contents

Abstract **1**

1 Problem Statement **3**

1.1 Objectives . . . 3

1.2 System Requirements . . . 3

2 Methodology **3**

2.1 Anomaly Scoring (Stage 3) . . . 4

2.2 Temporal Association and Ranking (Stage 6) . . . 4

2.3 Evaluation Protocol . . . 5

2.4 Results . . . 5

2.5 Limitations . . . 7

3 UI Design **7**

4 Image Enhancement and Filtering Techniques **9**

4.1 Illumination Normalisation . . . 9

4.2 Frequency-Domain Wave Estimate . . . 10

4.3 Gabor Filtering . . . 10

4.4 Uniform Local Binary Patterns . . . 10

4.5 Median Filtering and Colour Distance . . . 10

4.6 Glint Suppression by Chromaticity . . . 11

4.7 Morphological Filtering . . . 11

4.8 Mapping to Lab Sessions 1–5 . . . 11

5 VS Code for Functional Design **12**

5.1 Project Structure . . . 12

5.2 Running the Application . . . 12

5.3 Key Code . . . 13

References **14**

1 Problem Statement

The task is to give a rescue operator a short, ranked list of places to look in each aerial frame, so that a person in the water is not lost among waves, glitter and foam. Three things make this hard:

- **Small targets.** 618 of the 739 annotated people in the benchmark measure under 16 px (square root of box area).
- **Moving clutter.** Waves, sun glint, whitecaps and debris all differ from calm water, and all change from frame to frame.
- **No fixed appearance.** A partly submerged person can show only a head, and clothing colour varies.

1.1 Objectives

1. Return at most k candidate regions per frame, each with a confidence value.
2. Work without training data, using classical operations only.
3. Expose every intermediate stage and parameter, so an operator can inspect and tune the system.
4. Measure performance with recall@k, false alarms per frame, precision and stage-by-stage ablation.

1.2 System Requirements

Table 1: Functional, non-functional, software and input requirements.

Type	Requirement	How the project meets it
Functional	Accept video, an image folder or a single image	`run` command and GUI file loaders
Functional	Output a ranked top-k shortlist as an overlay and a CSV	`run --output --csv`; GUI shortlist table
Functional	Let the operator inspect stages and tune parameters	GUI stage selector, 16 sliders, config save and load
Functional	Evaluate against annotated data	`evaluate` command; COCO JSON and YOLO txt loaders
Non-functional	Run on a CPU with no GPU	About 2.7 frames/s at 1280×720 on a laptop CPU
Non-functional	Be repeatable and tested	40 passing unit and integration tests
Software	Python with NumPy $\geq$ 1.24, OpenCV $\geq$ 4.8, Pillow $\geq$ 9.0, Matplotlib $\geq$ 3.7, pytest $\geq$ 7.0; Tkinter for the GUI	`requirements.txt`
Input	Daylight aerial RGB imagery of open water	Night and thermal imagery are out of scope

2 Methodology

The method describes normal sea texture from the frame itself and flags what deviates, so it needs no labelled training images. Stages 1 to 5 run on each frame, and stage 6 uses the history across frames. Figure 1 shows the data flow.

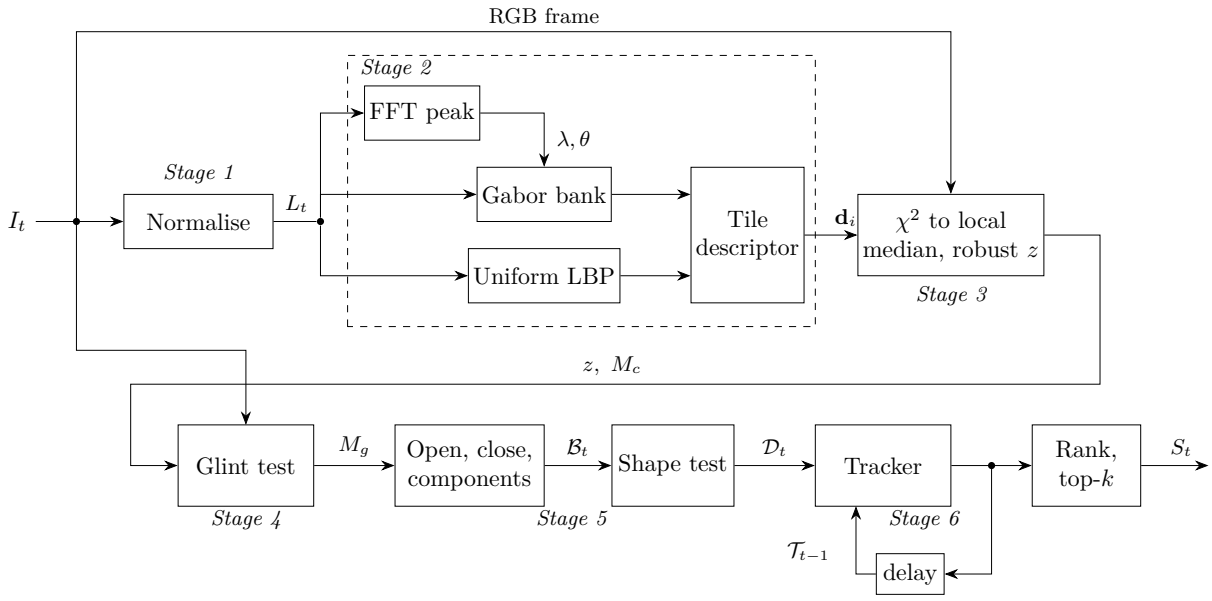


Figure 1: Block diagram of the pipeline. I_t: input frame; L_t: normalised lightness; λ, θ: dominant wavelength and orientation; $\mathbf{d}_i$: descriptor of tile i; z: anomaly map; M_c, M_g: candidate mask before and after the glint test; $\mathcal{B}_t$: blobs; $\mathcal{D}_t$: detections; $\mathcal{T}_{t-1}$: tracks from the previous frame; S_t: shortlist. Optional camera-motion compensation is not drawn.

The filtering operations in stages 1, 2, 4 and 5 are detailed in Section 4. This section covers how they are combined into a decision.

2.1 Anomaly Scoring (Stage 3)

Each overlapping tile has a descriptor made of mean Gabor energies and a 59-bin uniform LBP histogram [7]. The reference for normal sea is the median descriptor over 9×9-tile windows. A median stays clean when a person or a boat covers some tiles, and the local window adapts to near and far field.

Each tile is compared with its reference using the chi-square distance, computed separately for LBP and for reference-normalised Gabor energy:

$$\chi^2(a, b) = \frac{1}{2} \sum_i \frac{(a_i - b_i)^2}{a_i + b_i} \tag{1}$$

The two distances are put on one scale with a robust z-score and averaged with equal weight:

$$z = \frac{d - \text{median}(d)}{1.4826 \cdot \text{MAD}(d)} \tag{2}$$

Each pixel takes the maximum z of the tiles covering it, and the anomalous region is $z > 3$. Inside it, candidate pixels are those whose Lab colour differs from the median-filtered local sea colour by more than median $+ 4\sigma_{\text{MAD}}$.

2.2 Temporal Association and Ranking (Stage 6)

Detections are linked between frames by greedy nearest neighbour around a constant-velocity prediction, within a 40 px radius. A track is promoted after 5 hits if its drift is coherent, meaning

$\exp(-\text{spread}/6\,\text{px}) \geq 0.25$, where spread is the RMS deviation of step velocities from their mean. Gaps of up to 3 frames are tolerated, which covers a head briefly under a wave.

Shown tracks are ranked by this confidence, then multiplied by 0.8 per missed frame:

$$\text{conf} = 0.4 \cdot \text{conf}_{\text{anomaly}} + 0.6 \cdot \frac{\min(\text{hits}, 10)}{10} \cdot \text{coherence} \tag{3}$$

2.3 Evaluation Protocol

The benchmark has 6 rendered sequences of 60 frames at 1280×720 and 10 fps, with 739 annotated people. It covers altitudes of 50–150 m, calm to rough sea, strong sun glitter, a dark-clothed person and a panning camera.

A prediction counts as a hit when its centre lies within $\max(8\,\text{px}, \max(w, h))$ of the ground-truth centre, because IoU is very harsh on boxes of 10–30 px. Steady-state figures leave out the first 4 frames of each sequence, when the tracker cannot yet have promoted anything.

2.4 Results

The full system found 92.4% of people in a five-item shortlist with 1.77 false alarms per frame. **All results below are from the synthetic benchmark.** They show that the pipeline and evaluation work end to end, and they are not a claim about real-world performance.

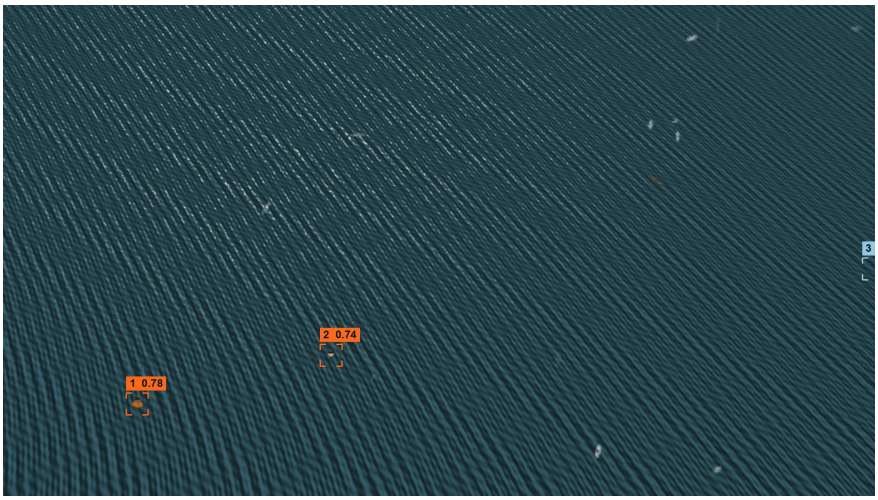


Figure 2: Output on sequence 5, frame 31. Orange brackets mark hits, and ranks and confidences are shown beside them.

Table 2: Ablation at $k = 5$. FA: false alarms; SS: steady state.

Run	Recall@5	Precision@5	FA/frame	SS recall@5	SS FA/frame	Frames/s
Full system	**92.4%**	**51.7%**	**1.77**	**99.3%**	**1.90**	**2.67**
Without temporal stage	89.2%	40.2%	2.73	88.7%	2.80	2.64
Without glint suppression	92.4%	51.2%	1.81	99.3%	1.94	2.75
Without morphology and shape filter	92.4%	45.7%	2.50	99.3%	2.68	2.65

Switching off one stage at a time shows what each contributes:

- **Temporal stage:** false alarms per frame fall 35%, from 2.73 to 1.77, and precision rises from 40.2% to 51.7%.
- **Morphology and shape filter:** false alarms per frame fall from 2.50 to 1.77.
- **Glint suppression:** only a small change here, from 1.81 to 1.77. The synthetic glitter specks are 1–3 px, so opening and the minimum-area limit remove most of them first.

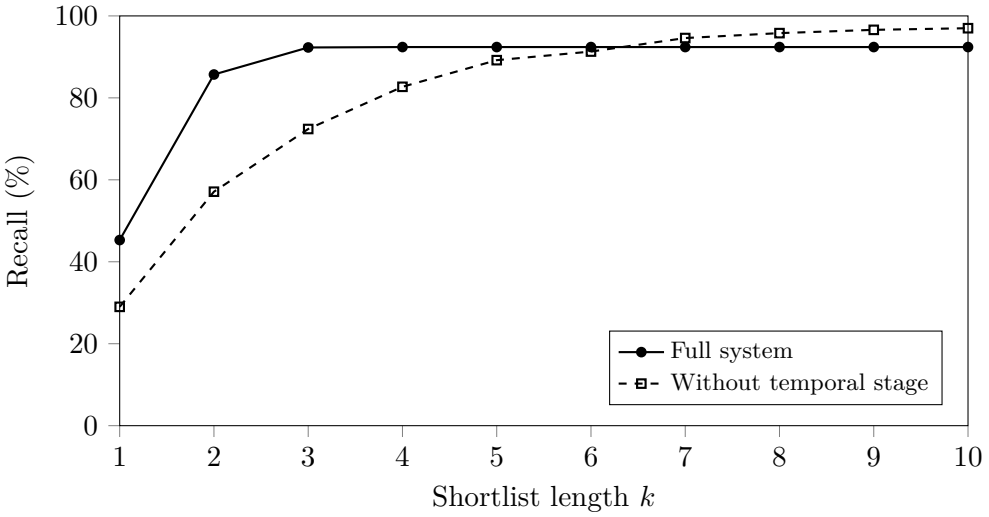


Figure 3: Recall against shortlist length on the synthetic benchmark (360 frames, 739 annotated people).

Persistence lifts people above short-lived clutter in the ranking, so recall@3 rises from 72.4% to 92.3% (Figure 3). The cost is the warm-up: nothing can be shown in a sequence's first 4 frames, so the full system stays at 92.4% and the run without tracking overtakes it at $k \geq 7$. At $k = 5$ the steady-state recall of the full system is 99.3%. Most frames hold 2–3 people, so recall@1 cannot exceed about 45%.

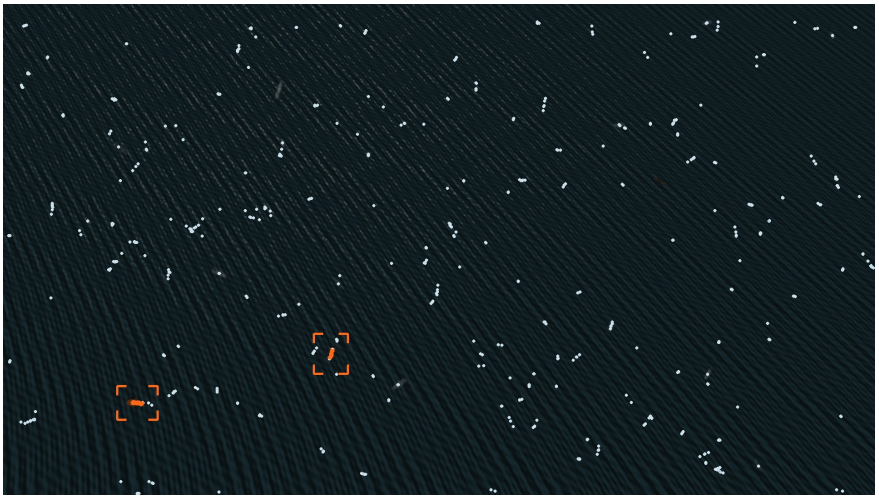


Figure 4: All detections across 60 frames of sequence 5. The two people form continuous tracks (orange); everything else appears briefly (pale dots).

Figure 4 shows why tracking works. The sequence produced 665 raw detections, yet each person's track stayed on the shortlist for 56 frames, and no other track lasted more than 7.

Recall@5 stayed steady across conditions, at 93.2% below 60 m, 92.9% at 60–100 m, 89.0% at 100–140 m and 93.3% at 140 m and above. It was 92.2% for targets under 16 px ($n = 618$) and 93.4% for 16–32 px ($n = 121$).

We inspected all 11 false alarms at frame 31 across the six sequences by eye: 3 looked like debris, 3 like whitecaps and 5 showed nothing visible. Debris is a real anomaly but not a person. One of the few steady-state misses is sequence 3, frames 27–30, where a person has a whitecap right beside them; that the whitecap is the cause has not been confirmed.

2.5 Limitations

- **Synthetic data only.** Real-world performance still has to be measured. Loaders are ready for SeaDronesSee [8] and MOBDrone [9] (COCO JSON) and AFO [10] (YOLO txt).
- **Dark, colourless targets** give weak colour and texture contrast.
- **Very bright white or grey clothing** can be removed by the glint test.
- **Non-sea content** such as a coastline or large ship can contaminate the local reference.
- **Persistent clutter**, such as long-lived whitecaps or debris, can be promoted to a track.
- **Warm-up:** nothing is shown in the first 4 frames of a sequence.
- **Camera motion:** phase correlation handles translation only, not rotation or zoom.
- **Hand-tuned parameters** may need retuning for a new camera or sea state.
- **Night and thermal imagery** are out of scope, because the colour test assumes daylight RGB.

3 UI Design

The operator GUI, built with Tkinter, puts the frame in the centre, the controls on the left and the ranked answer on the right. An operator reads the shortlist first, and a developer can open any stage to see why a box did or did not appear.

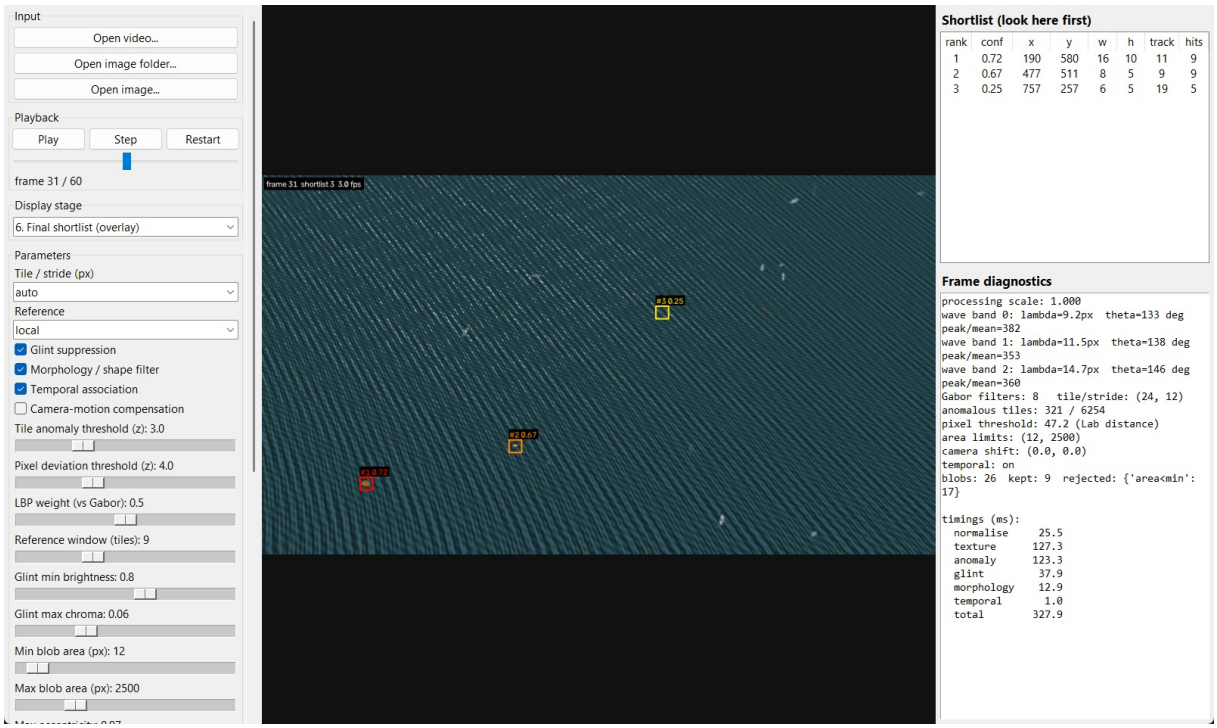


Figure 5: Main window on the final overlay, sequence 5, frame 31. The shortlist table gives rank, confidence, box, track id and hits.

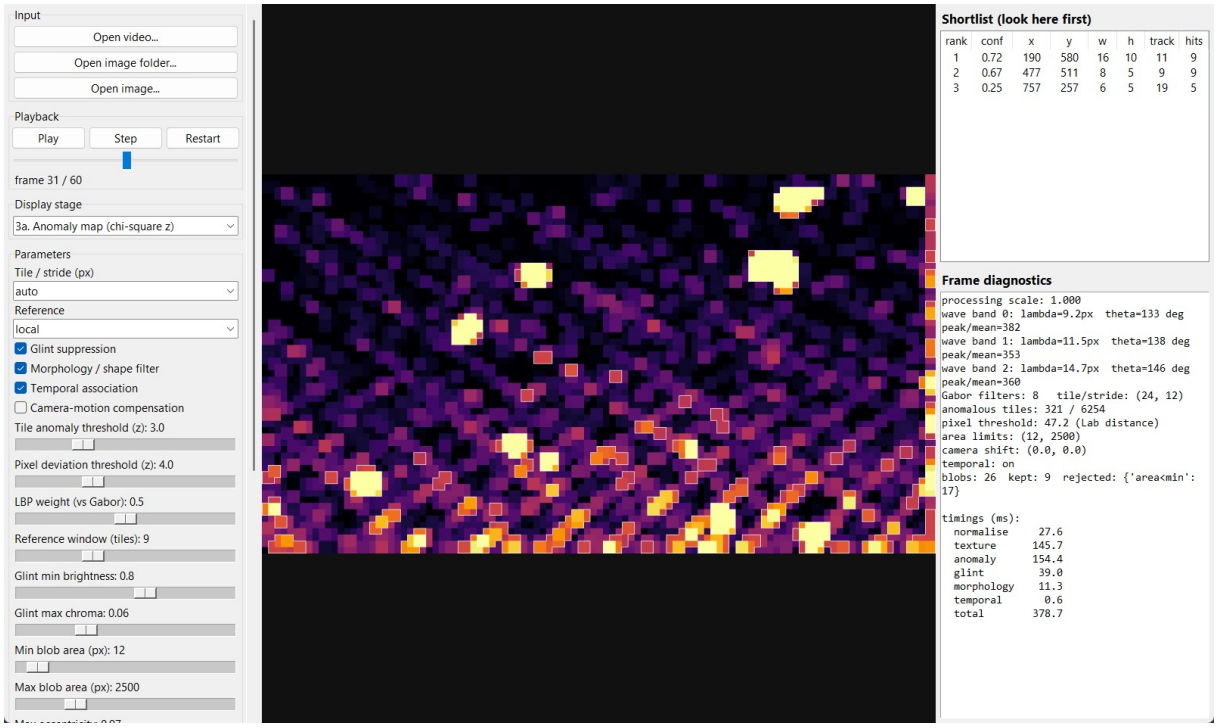


Figure 6: The same frame with the display stage set to 3a, the anomaly map. Parameters and diagnostics stay in place.

Table 3: GUI panels.

Panel	Controls	Purpose
Input (left)	Open video, Open image folder, Open image	Load any supported source
Playback (left)	Play, Step, Restart, frame slider	Move through a sequence or inspect one frame
Display stage (left)	Dropdown of 10 views, from input to final overlay	Show any intermediate result
Parameters (left)	Tile and reference dropdowns, stage on/off checkboxes, 16 sliders; Defaults, Save, Load	Tune live and keep configurations as JSON
Export (left)	Export annotated video	Save the overlay for review
Frame view (centre)	None	Display the selected stage with boxes, ranks and confidences
Shortlist (right)	Read-only table	Tell the operator where to look first
Frame diagnostics (right)	Read-only text	Show wave estimates, tile counts, thresholds, rejected blobs and per-stage timings

The frame in Figures 5 and 6 shows the diagnostics at work. 321 of 6,254 tiles were anomalous. Of 26 blobs, 17 were rejected as too small and 9 kept, and 3 tracks reached the shortlist.

4 Image Enhancement and Filtering Techniques

Every stage uses classical intensity, spatial, frequency or morphological operations [1], listed below in pipeline order.

4.1 Illumination Normalisation

These three steps run on the Lab lightness channel, so that texture statistics are comparable across the frame:

1. **Flat-fielding** divides lightness by a heavily Gaussian-smoothed copy of itself ($\sigma = 1/8$ of the frame width), which removes the smooth brightness gradient.
2. **CLAHE** equalises each region with its own histogram (clip limit 2.0, 8×8 regions).
3. **Piecewise-linear contrast stretch** maps the 1st and 99th percentiles to 10 and 245, through a lookup table.

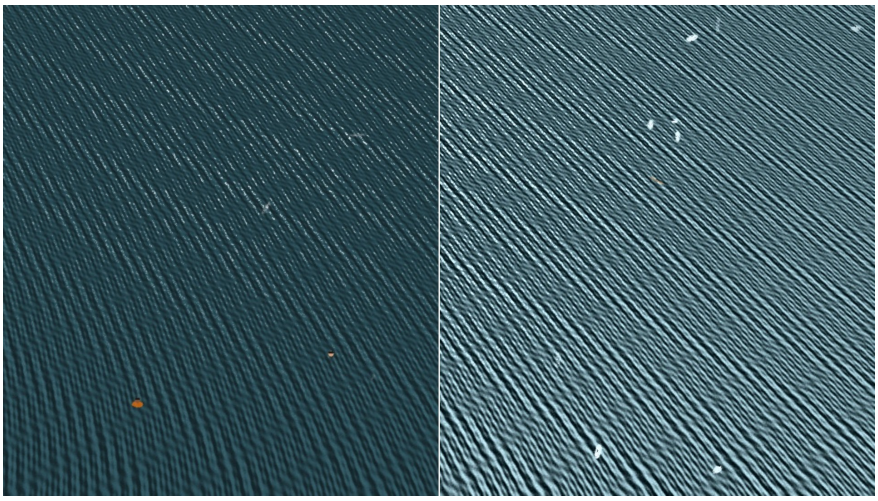


Figure 7: The same frame before (left) and after (right) normalisation.

4.2 Frequency-Domain Wave Estimate

The frame is split into 3 horizontal bands, and the 2-D DFT of a Hann-windowed patch gives each band's power spectrum. The strongest peak with a wavelength of 4–32 px gives the dominant wavelength and orientation. This tunes the Gabor filters to the sea actually in the frame.

4.3 Gabor Filtering

For each distinct band wavelength, the bank has 4 orientations: the dominant one, then +45°, +90° and +135°, with $\sigma = 0.56\lambda$ and $\gamma = 0.5$. Each filter runs as an even and odd pair, and the local energy $\sqrt{\text{even}^2 + \text{odd}^2}$ does not depend on wave phase.

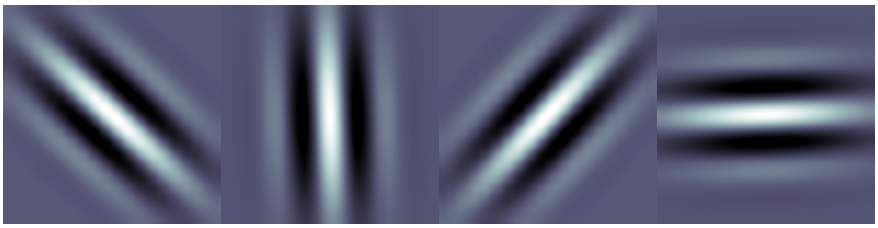


Figure 8: Even Gabor kernels at the four orientations of one wavelength.

4.4 Uniform Local Binary Patterns

Each pixel gets an 8-neighbour code (bit = neighbour $\geq$ centre), mapped to the 59 uniform bins [7]. A normalised LBP histogram per tile describes micro-texture without depending on absolute brightness. Tiles overlap by half, so a person on a tile border is not split.

4.5 Median Filtering and Colour Distance

The local sea colour is a median-filtered background. Pixels are measured against it in Lab space to turn an anomalous tile into a pixel-accurate candidate blob.

4.6 Glint Suppression by Chromaticity

Specular glint saturates all channels together, so its rgb chromaticity $(R, G, B)/(R+G+B)$ sits close to $(1/3, 1/3, 1/3)$. A pixel is glint if its mean is at least $0.8 \cdot 255$ and its distance from grey is at most 0.06. Requiring both keeps dark, colourless targets such as a black wetsuit.

4.7 Morphological Filtering

Opening (3×3 ellipse) removes specks, and closing (5×5) joins the fragments of a partly submerged body. Connected components (8-connectivity) must then pass three shape tests: area of 12–2,500 px, eccentricity ≤ 0.97 and solidity ≥ 0.45.

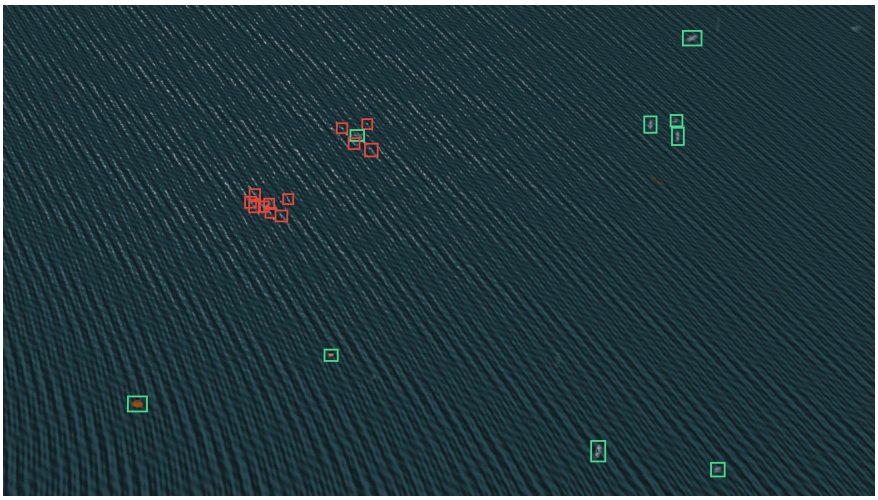


Figure 9: Blobs after morphology: green passed the shape filter and red were rejected.

4.8 Mapping to Lab Sessions 1–5

Table 4: Lab techniques and where each is implemented.

Technique	Implemented in
Intensity transform: piecewise-linear contrast stretch	`normalise.piecewise_linear`
Histogram processing: CLAHE	`normalise.normalise`
Spatial filtering: Gaussian, median, Gabor convolution	`normalise.flat_field`, `anomaly.colour_deviation`, `texture.gabor_energy`
Frequency domain: 2-D DFT power spectrum	`texture.estimate_wave`
Texture: uniform LBP, chi-square histogram distance	`texture.uniform_lbp`, `anomaly.chi_square`
Colour spaces: Lab, normalised rgb chromaticity	`anomaly.colour_deviation`, `glint.chromaticity`
Thresholding: robust median + k·MAD	`anomaly.score_anomalies`
Morphology: opening, closing, structuring elements	`morphology.refine`
Connected components, moments, convex hull	`morphology.refine`, `eccentricity`, `solidity`

5 VS Code for Functional Design

The functional design is a Python package, `sarvision`, with one module per stage and one shared configuration, so the GUI, command line and evaluation all run the same pipeline. The package is about 2,900 lines, and all 40 tests pass.

5.1 Project Structure

Table 5: Modules of the `sarvision` package.

File	Responsibility
`config.py`	Every tunable parameter, shared by GUI, CLI and evaluation
`normalise.py`	Stage 1: flat-fielding, CLAHE, contrast stretch
`texture.py`	Stage 2: wave spectrum, Gabor bank, uniform LBP, tile descriptors
`anomaly.py`	Stage 3: robust reference, chi-square distances, z-score fusion, candidate pixels
`glint.py`	Stage 4: chromaticity glint test
`morphology.py`	Stage 5: opening, closing, connected components, shape filter
`tracking.py`	Stage 6: association, persistence, drift test, ranking
`pipeline.py`	Runs the stages in order; optional camera-motion compensation
`gui.py`, `viz.py`	Tkinter operator GUI; stage rendering and overlay
`datasets.py`, `metrics.py`, `evaluate.py`	Dataset loaders, metrics, ablations, plots
`synthetic.py`	Synthetic sea sequences with ground truth
`tests/`	Unit tests for each stage and end-to-end tests

5.2 Running the Application

```
pip install -r requirements.txt
python -m sarvision synth --out data/synthetic
python -m sarvision gui data/synthetic/seq02.mp4
python -m sarvision run --input data/synthetic/seq02.mp4 --output results/demo/seq02_annotated.
    mp4 --csv results/demo/seq02_shortlist.csv
python -m sarvision evaluate --coco data/synthetic/annotations.json --ablations full,
    no_temporal,no_glint,no_morphology --out results/synthetic
python -m pytest -q
```

5.3 Key Code

Stage 1: normalisation (normalise.py)

```
def normalise(bgr: np.ndarray, cfg: Config) -> tuple[np.ndarray, np.ndarray]:
    """Return (normalised BGR frame, normalised grey frame), both uint8."""
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
    l_chan, a_chan, b_chan = cv2.split(lab)
    l_flat = flat_field(l_chan, cfg.flatten_sigma)
    clahe = cv2.createCLAHE(clipLimit=cfg.clahe_clip, tileGridSize=(cfg.clahe_grid, cfg.
    clahe_grid))
    l_eq = clahe.apply(l_flat)
    l_st = contrast_stretch(l_eq, cfg)
    bgr_n = cv2.cvtColor(cv2.merge([l_st, a_chan, b_chan]), cv2.COLOR_LAB2BGR)
    return bgr_n, l_st
```

Stage 2: Gabor energy and LBP codes (texture.py)

```
def gabor_energy(gray: np.ndarray, filt: GaborFilter) -> np.ndarray:
    """Local energy sqrt(even^2 + odd^2): phase-invariant response magnitude."""
    f = gray.astype(np.float32)
    e = cv2.filter2D(f, cv2.CV_32F, filt.even, borderType=cv2.BORDER_REFLECT)
    o = cv2.filter2D(f, cv2.CV_32F, filt.odd, borderType=cv2.BORDER_REFLECT)
    return cv2.magnitude(e, o)

def lbp_codes(gray: np.ndarray) -> np.ndarray:
    """Basic 8-neighbour LBP: bit i is 1 if neighbour i >= centre."""
    g = gray.astype(np.int16)
    p = np.pad(g, 1, mode="edge")
    h, w = g.shape
    code = np.zeros((h, w), dtype=np.uint8)
    for bit, (dy, dx) in enumerate(_NEIGHBOURS):
        nb = p[1 + dy:1 + dy + h, 1 + dx:1 + dx + w]
        code |= ((nb >= g).astype(np.uint8) << bit)
    return code
```

Stage 3: distance and robust score (anomaly.py)

```
def chi_square(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """0.5 * sum((a - b)^2 / (a + b)) over the last axis."""
    return 0.5 * np.sum((a - b) ** 2 / (a + b + EPS), axis=-1)

def robust_z(x: np.ndarray) -> np.ndarray:
    med = np.median(x)
    mad = np.median(np.abs(x - med)) * MAD_TO_SIGMA
    if mad < EPS:
        mad = x.std() + EPS
    return (x - med) / mad
```

Stage 4: glint mask (`glint.py`)

```
def glint_mask(bgr: np.ndarray, cfg: Config) -> np.ndarray:
    """Boolean mask of bright, achromatic pixels."""
    total = bgr.astype(np.uint16).sum(axis=-1)
    bright = total >= 3.0 * 255.0 * cfg.glint_value
    out = np.zeros(bright.shape, dtype=bool)
    if bright.any():
        px = bgr[bright]
        out[bright] = chroma_distance(px[None])[0] <= cfg.glint_chroma
    return out
```

Stage 5: opening and closing (`morphology.py`, excerpt from `refine`)

```
if cfg.open_ksize > 1:
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (cfg.open_ksize, cfg.open_ksize))
    work = cv2.morphologyEx(work, cv2.MORPH_OPEN, k)
if cfg.close_ksize > 1:
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (cfg.close_ksize, cfg.close_ksize))
    work = cv2.morphologyEx(work, cv2.MORPH_CLOSE, k)

n, labels, stats, cents = cv2.connectedComponentsWithStats(work, connectivity=8)
```

Stage 6: ranking (`tracking.py`)

```
def ranked(self, frame: int) -> list[RankedCandidate]:
    """Promoted tracks ranked by the combined anomaly/persistence confidence."""
    cfg = self.cfg
    out = []
    for t in self.tracks:
        coh = t.coherence(cfg)
        if t.hits < cfg.min_persistence or coh < cfg.min_coherence:
            continue
        missed = frame - t.last_frame
        persist = min(t.hits, cfg.persistence_horizon) / float(cfg.persistence_horizon) * coh
        conf = cfg.anomaly_weight * t.conf_ema + (1.0 - cfg.anomaly_weight) * persist
        conf *= 0.8 ** missed
        out.append(RankedCandidate(t.det, float(conf), t.tid, t.hits, coh, missed))
    out.sort(key=lambda c: -c.conf)
    return out
```

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Pearson Education, 2012.

[2] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2011.

[3] M. Sonka, V. Hlavac and R. Boyle, *Digital Image Processing and Computer Vision*, India ed. Cengage Learning, 2009.

[4] J. Howse, P. Joshi and M. Beyeler, *OpenCV: Computer Vision Projects with Python*. Packt Publishing, 2016.

[5] D. L. Baggio, S. Emami, D. M. Escrivá, K. Ievgen, N. Mahmood, J. Saragih and R. Shilkrot, *Mastering OpenCV 3*, 2nd ed. Packt Publishing, 2017.

[6] R. Laganiere, *OpenCV Computer Vision Application Programming Cookbook*, 2nd ed. Packt Publishing, 2014.

[7] T. Ojala, M. Pietikäinen and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 7, pp. 971–987, 2002. doi:10.1109/TPAMI.2002.1017623

[8] L. A. Varga, B. Kiefer, M. Messmer and A. Zell, "SeaDronesSee: A Maritime Benchmark for Detecting Humans in Open Water," in *Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022, pp. 2260–2270.

[9] D. Cafarelli et al., "MOBDrone: A Drone Video Dataset for Man OverBoard Rescue," in *Image Analysis and Processing – ICIAP 2022*. Springer, 2022, pp. 633–644.

[10] J. Gąsienica-Józkowy, M. Knapik and B. Cyganek, "An ensemble deep learning method with optimized weights for drone-based water rescue and surveillance," *Integrated Computer-Aided Engineering*, vol. 28, no. 3, pp. 221–235, 2021. doi:10.3233/ICA-210649